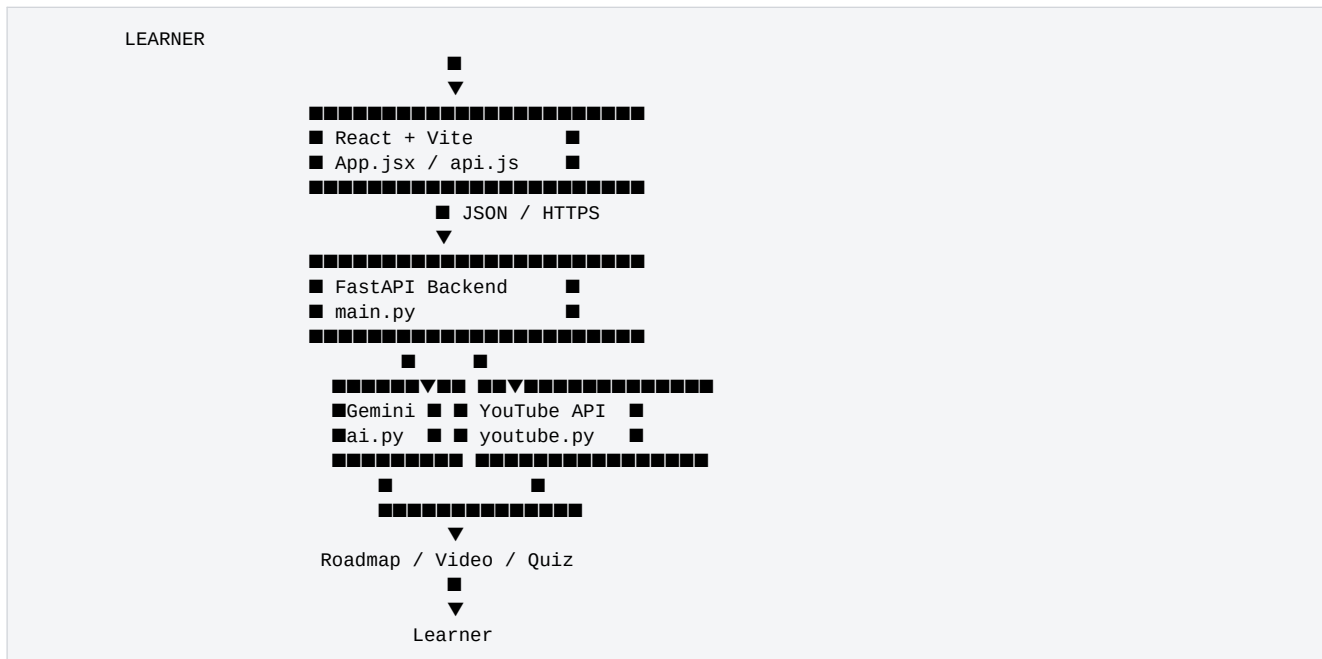




## Main modules

| File                   | Responsibility                                            |
|------------------------|-----------------------------------------------------------|
| frontend/src/App.jsx   | Complete user interaction flow and React state.           |
| frontend/src/api.js    | POST helper and /roadmap, /youtube, /quiz functions.      |
| frontend/src/style.css | Responsive layout and visual styling.                     |
| backend/main.py        | FastAPI app, CORS, Pydantic request models, endpoints.    |
| backend/ai.py          | Gemini client, JSON extraction, roadmap and quiz prompts. |
| backend/youtube.py     | Search, details, comments, ranking and recommendation.    |

## 4. Frontend Workflow

App.jsx implements the product as a sequential experience. The interface first collects the profile, then reveals the roadmap, then the selected topic's resource, and finally the quiz.

### User journey

```

Profile
↓
createRoadmap()
↓
getRoadmap(profile)
↓
Select roadmap step
↓
getVideo(topic, profile)
↓
Recommended video + feedback samples
↓
getQuiz(topic, profile)
↓
Submit answers
↓
Score + next-step message
  
```

## Important UI states

- loading="roadmap" while generating the roadmap.
- loading="video" while finding a resource.
- loading="quiz" while generating the quiz.
- error displays request failures without silently failing.
- score stores the local quiz result.

## Frontend API base URL

frontend/src/api.js

```
const API = import.meta.env.VITE_API_URL || "http://localhost:8000";
```

## 5. AI Roadmap Generation

The roadmap prompt is designed as a structured planning task. Gemini receives the full learner profile and is explicitly asked to avoid repeating known skills unless they are required prerequisites.

Requested roadmap contract

```
{
  "goal": "string",
  "summary": "short explanation",
  "roadmap": [
    {
      "step": 1,
      "title": "string",
      "description": "string",
      "why": "why this is recommended for this learner",
      "difficulty": "Beginner|Intermediate|Advanced",
      "estimated_hours": 5,
      "prerequisites": [],
      "milestone": "string"
    }
  ]
}
```

### Why the prompt is effective

- It gives the AI concrete learner context instead of only a topic name.
- It asks for ordered steps, making the result actionable.
- The “why” field makes the recommendation explainable.
- Estimated hours connect the path to time availability.
- Prerequisites and milestones make each step more than a title.

### Implementation detail

ai.py uses google.genai, requests JSON output with response\_mime\_type="application/json", strips optional Markdown code fences, and parses the result with json.loads. The current code does not perform a second schema validation after parsing.

## 6. YouTube Recommendation Engine

The YouTube layer is a particularly useful part of the prototype because it goes beyond displaying a search result. It searches for “topic + experience-level tutorial”, retrieves statistics, retrieves comments where available, computes a score, and sorts the candidates.

Current scoring logic

```
relevance = 100 if topic appears in title/description else 65

engagement =
  0.65 * normalized like ratio
  + 0.35 * normalized comment ratio

comment_bonus = 10 if comments exist else 0

final_score =
  0.55 * relevance
  + 0.35 * engagement
  + 0.10 * comment_bonus
```

### What makes this defensible

- The ranking is explicit and inspectable rather than a black box.

- Experience level affects the search query.
- Engagement is normalized instead of relying only on raw view counts.
- The selected score is displayed in the UI.
- The system can show real learner comments as context.

## Be accurate in the submission

The current code does **not** perform AI sentiment analysis of comments. It uses comment availability as a bonus and displays up to five feedback samples. The README describes comment sentiment/ranking as a future extension.

## 7. Assessment Layer

The quiz prompt asks Gemini for exactly 3–10 single-answer multiple-choice questions, while the frontend requests five. Each question contains four options, a zero-based correct-answer index, and an explanation.

Quiz contract

```
POST /quiz
{
  "topic": "...",
  "profile": {...},
  "count": 5
}
```

### Current scoring behavior

The browser compares each selected option index against q.answer. A score of at least 70% displays a move-forward message; otherwise the learner is told to review the topic and try again.

### Assessment-aware vs fully adaptive

This distinction is important: the prototype is **assessment-aware** because the learner receives score-based guidance. It is not yet fully **adaptive** because the score does not automatically modify the roadmap, persist weak areas, or regenerate the next learning step.

## 8. Backend API Reference

| Endpoint | Method | Request        | Purpose                                     |
|----------|--------|----------------|---------------------------------------------|
| /        | GET    | None           | Returns a simple API-running message.       |
| /roadmap | POST   | LearnerProfile | Generates personalized roadmap.             |
| /youtube | POST   | TopicRequest   | Returns recommended video and alternatives. |
| /quiz    | POST   | QuizRequest    | Generates topic-specific quiz.              |

### Data models

backend/main.py

```
class LearnerProfile(BaseModel):
    goal: str
    skills: list[str] = []
    experience: str = "Beginner"
    hours_per_week: int = 5
    interests: list[str] = []

class TopicRequest(BaseModel):
    topic: str
    profile: LearnerProfile

class QuizRequest(BaseModel):
    topic: str
    profile: LearnerProfile
    count: int = 5
```

### Backend design advantage

Pydantic models establish a clear JSON contract at the API boundary. This is a good foundation for later schema validation, authentication, persistence, and observability.

## 9. Local Setup & Execution

The following text is ready to paste into the HackerEarth “Local setup & execution instructions” field:

HackerEarth-ready instructions

```

Prerequisites:
- Python 3.10+
- Node.js 18+
- npm
- Git

1. Clone:
git clone https://github.com/keshavreddyaletip/AI-Learning-Path.git
cd AI-Learning-Path

2. Backend:
cd backend
python -m venv venv

Windows:
venv\Scripts\activate

macOS/Linux:
source venv/bin/activate

pip install -r requirements.txt
copy .env.example .env
# macOS/Linux: cp .env.example .env

3. Configure backend .env:
AI_API_KEY=<your Gemini API key>
YOUTUBE_API_KEY=<your YouTube Data API key>
AI_MODEL=<currently supported Gemini model>

4. Start backend:
uvicorn main:app --reload

5. Open a second terminal:
cd frontend
npm install
npm run dev

6. Open the Vite URL, normally:
http://localhost:5173

The backend already allows http://localhost:5173 through CORS
in the submitted snapshot.

```

## Security

Keep both API keys on the backend. Never commit .env or expose provider secrets through frontend code.

# 10. Deployment & Production Considerations

| Layer    | Check                                                                |
|----------|----------------------------------------------------------------------|
| Frontend | VITE_API_URL points to the deployed backend.                         |
| Backend  | Render service starts successfully and exposes the API.              |
| CORS     | The exact production Vercel origin is allowed.                       |
| Gemini   | API key and currently supported model are configured on the backend. |
| YouTube  | API key is valid and quota is available.                             |
| Secrets  | Keys exist only in server-side environment variables.                |

## CORS issue diagnosis

The submitted main.py explicitly permits the deployed Vercel origin and localhost:5173. If a different Vercel production/preview URL is used, the backend allow-list must be updated. The browser's OPTIONS preflight request should be checked before debugging the AI provider.

Archived CORS configuration

```
allow_origins=[
    "https://ai-learning-path-inky.vercel.app",
    "http://localhost:5173",
],
allow_methods=["GET", "POST", "OPTIONS"],
allow_headers=["Content-Type", "Accept", "Origin"],
```

## 11. Gemini Configuration Note

The uploaded snapshot contains AI\_MODEL=gemini-2.5-flash in both ai.py's default and .env.example. Because the deployment issue discussed previously reported that this model identifier was unavailable for the relevant API access, the final production configuration should use the currently supported model actually working with the project's API account.

Recommended environment pattern

```
AI_API_KEY=<server-side secret>
YOUTUBE_API_KEY=<server-side secret>
AI_MODEL=<currently supported model>
```

Do not hard-code a model value across the application. Keeping it in the environment makes provider/model migrations safer and faster.

## 12. What Makes the Existing Code Stand Out

Without changing the code, the strongest differentiators are already present. The submission should make these visible to the evaluator instead of burying them under a generic “AI project” description.

| Existing strength                | How to present it                                              |
|----------------------------------|----------------------------------------------------------------|
| Profile-aware AI                 | “The roadmap is generated for a learner, not just a topic.”    |
| Structured roadmap               | Highlight why, prerequisites, hours and milestones.            |
| Transparent recommendation score | Explain exactly how the YouTube candidate is ranked.           |
| Real external data               | Show YouTube statistics and learner feedback samples.          |
| Assessment                       | Demonstrate that learning does not stop at resource discovery. |
| Deployment                       | Mention the real Vercel/Render integration and debugging work. |

### Recommended narrative

Introduce one fictional learner and demonstrate the entire journey: **Goal** → **Profile** → **Roadmap** → **Topic** → **Recommended Resource** → **Quiz** → **Score** → **Next Action**. This turns a feature list into a story.

## 13. Challenges & Solutions

| Challenge                    | Solution / lesson                                                                |
|------------------------------|----------------------------------------------------------------------------------|
| CORS preflight               | Explicit backend CORS configuration for the frontend origin.                     |
| Gemini model availability    | Move model selection into environment configuration and verify supported models. |
| Local vs production behavior | Separate VITE_API_URL, backend environment variables, and deployment settings.   |
| Resource overload            | Use a ranking heuristic instead of blindly displaying the first YouTube result.  |
| AI output reliability        | Request structured JSON and parse the response.                                  |

### Why include failures?

A professional submission can show that the system was tested beyond the happy path. CORS, external model availability, environment configuration, and deployment behavior are real engineering concerns.

## 14. Honest Limitations & Future Roadmap

| Current                        | Future extension                                |
|--------------------------------|-------------------------------------------------|
| Session-only state             | Persist learner profile, progress and scores.   |
| Score changes only the message | Use weak topics to adapt the next roadmap step. |



| Current                                | Future extension                                |
|----------------------------------------|-------------------------------------------------|
| Comment availability bonus             | Add quality/sentiment analysis if desired.      |
| JSON parsing without schema validation | Validate AI output with Pydantic models.        |
| Raw exception detail can reach client  | Use safe user messages + server logs.           |
| No authentication                      | Add accounts if the product becomes multi-user. |

### Important: do not overclaim

The current implementation should be described as a focused personalized learning prototype with assessment and recommendation components. Avoid claiming persistent adaptive learning, AI comment sentiment, or long-term learner analytics until those are actually implemented.

## 15. Evaluator-Focused Demonstration Plan

| Time      | Demonstration                                                           |
|-----------|-------------------------------------------------------------------------|
| 0:00–0:25 | State the learner problem: too many resources, no clear direction.      |
| 0:25–1:10 | Enter one learner profile and generate the roadmap.                     |
| 1:10–1:50 | Highlight ordered steps, rationale, hours, prerequisites and milestone. |
| 1:50–2:35 | Select a topic and show the ranked YouTube recommendation + score.      |
| 2:35–3:20 | Show feedback samples and generate the quiz.                            |
| 3:20–4:00 | Submit answers and demonstrate score-based next action.                 |
| 4:00–4:30 | Briefly show architecture and deployment stack.                         |

### Opening line

**“The internet doesn't have a shortage of learning content. It has a shortage of direction. AI Learning Path is designed to provide that direction.”**

### Closing line

**“Don't give learners more resources. Give them a direction.”**

## 16. Final Technical Assessment

The uploaded submission is a coherent, compact prototype with a clear end-to-end concept. It demonstrates frontend state management, a typed FastAPI boundary, AI integration, external API integration, recommendation heuristics, structured generation, and assessment.

**Its strongest technical story is the combination of three stages:**

Existing product loop

1. PLAN → AI creates a personalized learning sequence
2. FIND → YouTube candidates are searched and ranked
3. CHECK → AI generates a quiz and the learner receives feedback

The most important presentation improvement is to make the evaluator notice that this is a **decision workflow**, not merely a text-generation demo. The code already supports that story.

### Submission-ready one-line description

“AI Learning Path converts a learner’s goal and profile into a structured learning journey, connects each step to a ranked real-world resource, and checks understanding through AI-generated assessment.”

### Final note

This document was prepared by directly inspecting submission\_1.zip. It intentionally distinguishes implemented functionality from future ideas so the submission remains technically credible.